

Méthodes synchronisées en Java

Fabrice.Kordon@lip6.fr



En guise d'introduction...

2

Créer des sections critiques par synchronisation

-  Retarder des séquences de code
-  Laisser d'autres séquences de code progresser
-  Exclusion mutuelle?

Il existe de nombreux mécanismes!

-  Certains génériques
-  D'autres associés à un/plusieurs langages(s)

Directive «synchronized» appliquées aux méthodes

-  Propre à Java
-  Ressemble à la notion de «point d'entrée» en Ada

Objectif de la directive synchronized

3

Garantir l'exclusion mutuelle

- Portée : l'exécution des méthodes synchronisées d'une classe
- Protège l'accès aux attributs via ces méthodes

Attention

- Cela ne porte que sur les méthodes protégées
- Il faut réfléchir
 - ▶ Verrouillage total?
 - ▶ Préférence au verrouillage des modifications?

Compteur partagé et protégé (classe CompteurProtege)

4

```
class CompteurProtege {  
  
    private String monNom;  
    private int compteur = 0;  
  
    CompteurProtege (String n) {  
        monNom = n;  
    }  
  
    synchronized public void inc() {  
        compteur++;  
        System.out.println ("après inc, " + this.toString());  
    }  
  
    synchronized public void dec() {  
        compteur--;  
        System.out.println ("après dec, " + this.toString());  
    }  
  
    public String toString () {  
        return monNom + "=" + compteur;  
    }  
}
```


Compteur partagé et protégé (classe Client)

5

```
class Client implements Runnable {
    private int monNumero;
    private CompteurProtege monCpt;
    Client (int i, CompteurProtege c) {
        monNumero = i;
        monCpt = c;
    }
    public void run() {
        System.out.println (monNumero + " est lancé");
        for (int i = 1 ; i <= 4 ; i++) {
            if (monNumero % 2 == 0) {
                monCpt.inc();
            } else {
                monCpt.dec();
            }
            try { // On met la dose pour forcer une commutation;-)
                Thread.sleep (100); // millisecondes
            }
            catch (InterruptedException ie) {
                System.err.println("Sleep impossible pour " + monNumero);
            }
        }
    }
    public String toString () {
        return "client " + monNumero;
    }
}
```


Compteur partagé et protégé (classe «principale»)

6

```
public class CompteurPartage2 {  
  
    public static void main(String []args) {  
        CompteurProtege cpt = new CompteurProtege("cpt");  
  
        Thread lesClients [] = new Thread [6];  
        for (int i = 0; i < lesClients.length ; i++) {  
            Client c = new Client (i+1, cpt);  
            lesClients [i] = new Thread(c);  
            lesClients[i].start();  
        }  
    }  
}
```


Compteur partagé et protégé (classe «principale»)

6

```
public class CompteurPartage2 {
```

```
$ java CompteurPartage2  
2 est lancé  
6 est lancé  
5 est lancé  
3 est lancé  
1 est lancé  
4 est lancé  
après inc, cpt=1  
après inc, cpt=2  
après dec, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après dec, cpt=-1  
après dec, cpt=-2  
après inc, cpt=-1  
après inc, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après inc, cpt=1  
après inc, cpt=2  
après dec, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après inc, cpt=1  
après inc, cpt=2  
après dec, cpt=1  
après dec, cpt=0
```

```
$ java CompteurPartage2  
4 est lancé  
5 est lancé  
1 est lancé  
après inc, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après inc, cpt=0
```

**L'entrelacement
est toujours là**

```
après inc, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après dec, cpt=-1
```

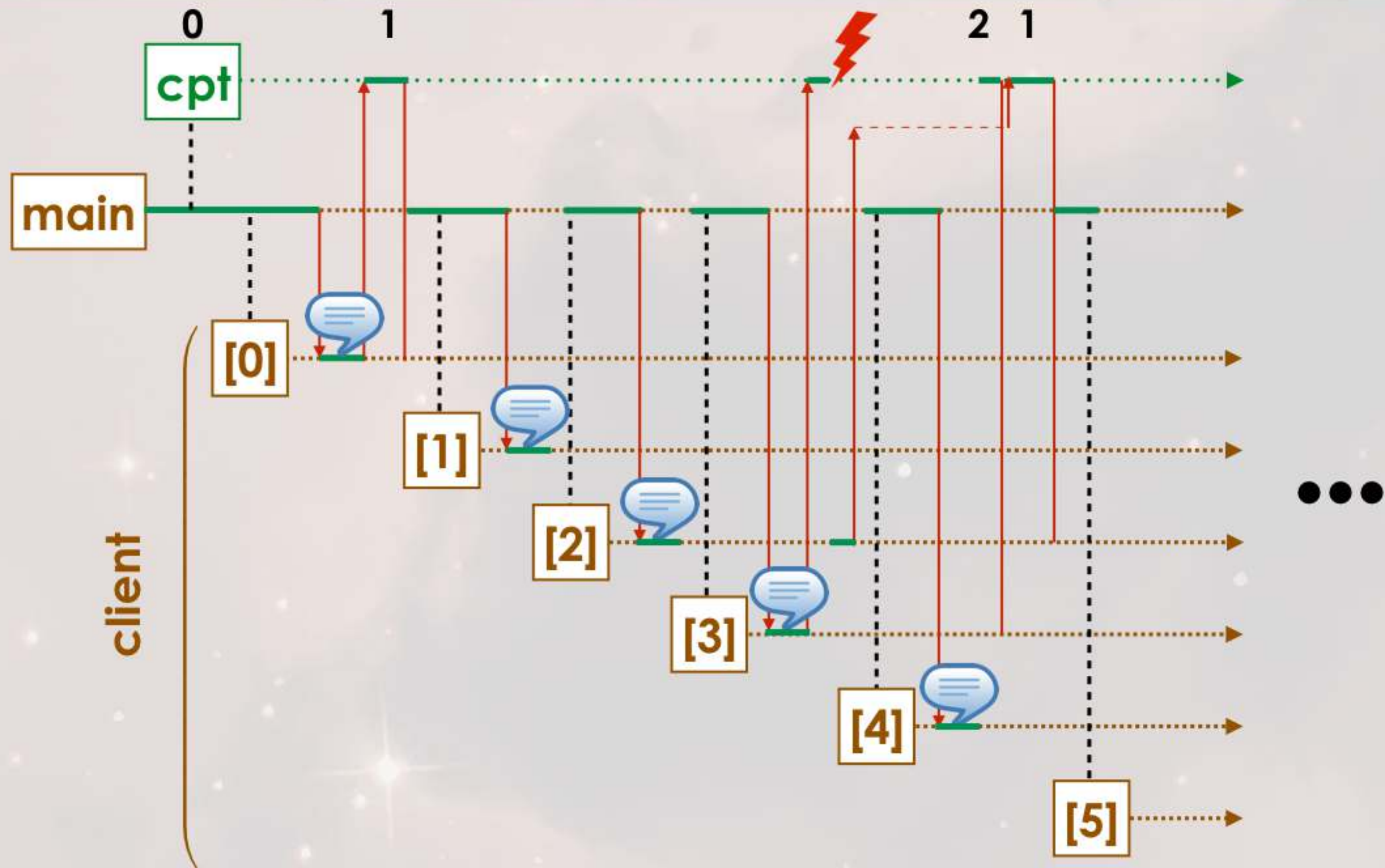
**Les problèmes
sont réglés**

```
après inc, cpt=2  
après dec, cpt=1  
après inc, cpt=2  
après dec, cpt=1  
après dec, cpt=0
```

```
$ java CompteurPartage2  
1 est lancé  
3 est lancé  
6 est lancé  
4 est lancé  
2 est lancé  
5 est lancé  
après dec, cpt=-1  
après dec, cpt=-2  
après inc, cpt=-1  
après inc, cpt=0  
après inc, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après dec, cpt=-2  
après dec, cpt=-3  
après inc, cpt=-2  
après inc, cpt=-1  
après inc, cpt=0  
après inc, cpt=1  
après inc, cpt=2  
après inc, cpt=3  
après dec, cpt=2  
après dec, cpt=1  
après dec, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après dec, cpt=-1  
après inc, cpt=0  
après inc, cpt=1  
après dec, cpt=0
```


Chronogramme du début d'une exécution possible

7

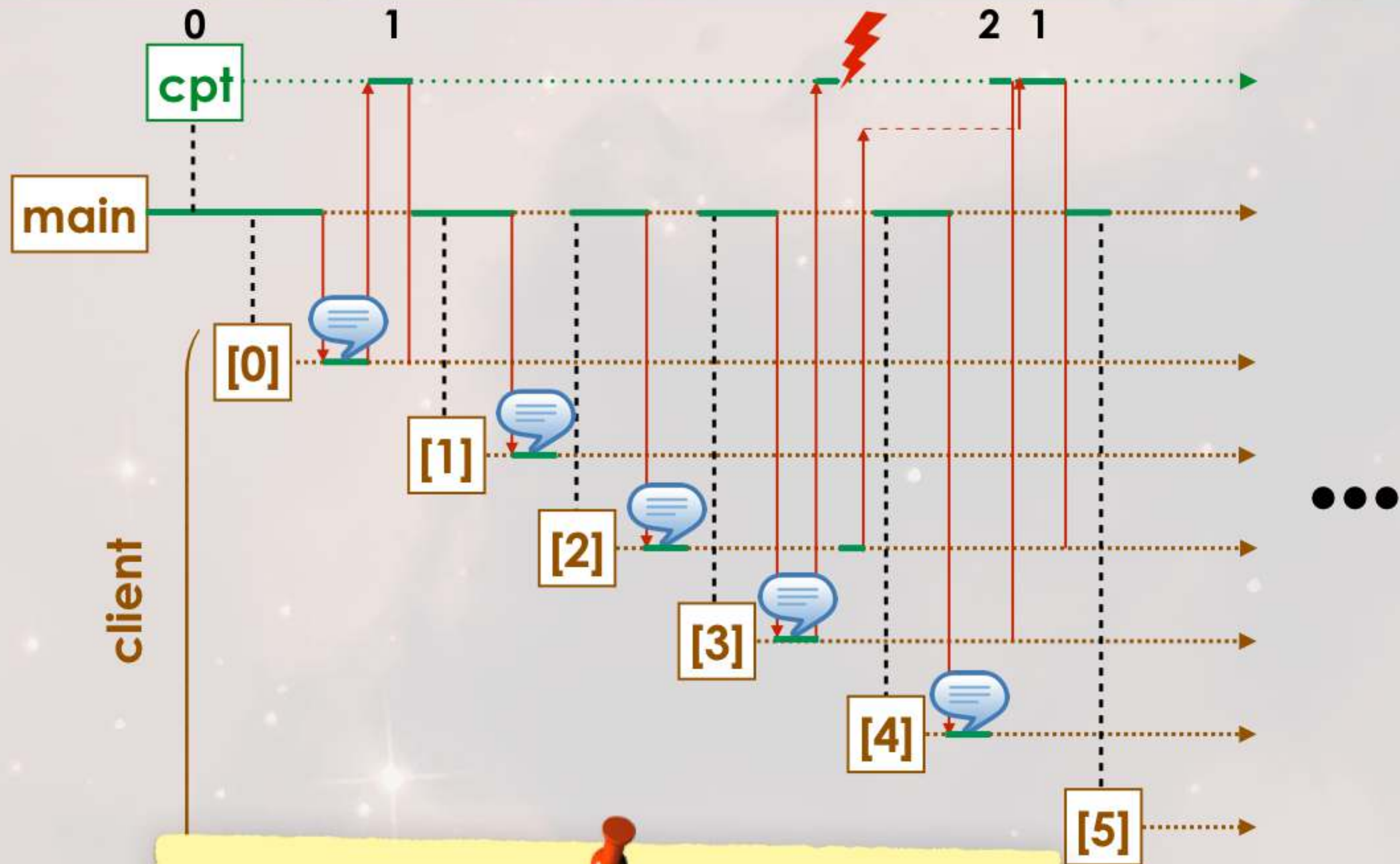


```
CompteurProtege cpt = new CompteurProtege("cpt");
Thread lesClients [] = new Thread [6];
for (int i = 0; i < lesClients.length ; i++) {
    Client c = new Client (i+1, cpt);
    lesClients [i] = new Thread(c);
    lesClients[i].start();
}
```

```
System.out.println (monNumero + " est lancé");
for (int i = 1 ; i <= 4 ; i++) {
    if (monNumero % 2 == 0) {
        monCpt.inc();
    } else {
        monCpt.dec();
    }
    ...
}
```


Chronogramme du début d'une exécution possible

7



Les objets ont un «verrou»
Dédié à la gestion du synchronise

```
CompteurProtege cpt;  
Thread lesClients [  
for (int i = 0; i < 5; i++) {  
    Client c = new Client(i);  
    lesClients[i] = c;  
    lesClients[i].start();  
}
```

```
    n (monNumero + " est lancé");  
    i <= 4; i++) {  
        % 2 == 0) {  
            nc();  
            ec();  
        }  
    }  
}
```